Report Laboratorio 3: Transformers

Emiliano Lorenzo

Gennaio 2026

Indice

1 Introduzione **3**
1.1 Contesto Tecnologico . . . 3
1.2 Caso di Studio: Sentiment Analysis . . . 3

2 Exploratory Data Analysis **4**
2.0.1 Distribuzione delle Classi . . . 4
2.0.2 Analisi della Lunghezza delle Sequenze . . . 4
2.1 Inizializzazione del Modello e Tokenizzazione . . . 5
2.1.1 Analisi del Processo di Tokenizzazione . . . 5
2.1.2 Analisi dell'Output (Hidden States) . . . 6
2.2 Costruzione della Baseline: Feature Extraction con SVM . . . 6
2.2.1 Estrazione delle Rappresentazioni Semantiche . . . 6
2.2.2 Addestramento e Valutazione del Classificatore . . . 6
2.3 Full Fine-Tuning di DistilBERT . . . 7
2.3.1 Setup Sperimentale . . . 7
2.3.2 Iperparametri e Training Strategy . . . 7
2.3.3 Analisi dei Risultati . . . 8

3 Parameter-Efficient Fine-Tuning (PEFT) con LoRA **8**
3.1 Fondamenti Teorici . . . 8
3.2 Configurazione Sperimentale e Risultati . . . 9
3.2.1 Configurazione FT2: Basic LoRA . . . 9
3.2.2 Configurazione FT3: Optimized LoRA . . . 9
3.2.3 Analisi Comparativa . . . 9
3.3 Ottimizzazione Bayesiana degli Iperparametri . . . 9
3.3.1 Spazio di Ricerca e Configurazione . . . 10
3.3.2 Analisi dei Risultati . . . 10

4 Adattamento tramite Prompt Tuning **11**
4.1 Configurazione e Iperparametri . . . 11
4.2 Analisi Critica dei Risultati . . . 11

5 Strategie di Congelamento Parziale e Confronto Finale **12**
5.1 Fine-Tuning Selettivo di DistilBERT 12
5.2 BERT Base e LoRA 12
5.3 Valutazione Comparativa sul Test Set 12

1 Introduzione

Il presente elaborato documenta l'attività laboratoriale volta all'esplorazione e all'utilizzo dell'ecosistema **HuggingFace** per l'adattamento di modelli linguistici pre-addestrati a specifici task di *Natural Language Processing* (NLP).

L'obiettivo principale del progetto è l'implementazione di una pipeline di *Sentiment Analysis*, costruita attraverso un approccio incrementale che parte dall'analisi dei dati fino al *fine-tuning* di modelli basati sull'architettura Transformer.

1.1 Contesto Tecnologico

Al centro di questa sperimentazione vi sono due componenti fondamentali dell'NLP moderno:

- **I Transformers:** Introdotti da Vaswani et al. nel 2017, i Transformers rappresentano l'attuale stato dell'arte per l'elaborazione del linguaggio naturale. A differenza delle precedenti reti ricorrenti (RNN o LSTM), questa architettura si basa interamente sul meccanismo di *Self-Attention*, che permette al modello di pesare l'importanza delle diverse parole in una frase simultaneamente, gestendo efficacemente le dipendenze a lungo raggio e consentendo una parallelizzazione massiva durante l'addestramento.

- **L'Ecosistema HuggingFace:** HuggingFace si è affermato come il punto di riferimento centrale per la comunità di Machine Learning. Esso fornisce non solo un vasto repository di modelli pre-addestrati (Model Hub), ma anche le librerie software (`transformers`, `datasets`, `peft`) che astraggono la complessità sottostante. Queste astrazioni permettono di manipolare agevolmente architetture complesse come BERT, GPT o RoBERTa, facilitando le operazioni di tokenizzazione, caricamento dei pesi e addestramento.

1.2 Caso di Studio: Sentiment Analysis

L'esperimento condotto (Exercise 1) si focalizza su un task di classificazione binaria supervisionata. Partendo da un modello **BERT** pre-addestrato (*bert-base-uncased*), abbiamo sviluppato un classificatore in grado di distinguere la polarità (positiva o negativa) di recensioni cinematografiche.

Il dataset utilizzato è il **Cornell Rotten Tomatoes movie review dataset**. Si tratta di un corpus bilanciato e di dimensioni contenute, ideale per la prototipazione rapida, composto da:

- 5.331 frasi etichettate come positive;
- 5.331 frasi etichettate come negative.

Nel corso dell'elaborato verranno dettagliate le fasi di ingestione dei dati tramite la libreria `datasets`, la gestione degli split per il training e la validazione, e l'investigazione delle meccaniche interne delle classi di HuggingFace necessarie per adattare il modello al nuovo dominio applicativo.

2 Exploratory Data Analysis

Prima di procedere con la fase di fine-tuning, è stata condotta un'analisi esplorativa sul dataset *Cornell Rotten Tomatoes* per comprenderne la struttura e le caratteristiche statistiche. Il dataset è stato convertito in un `DataFrame` Pandas per facilitare la manipolazione e la visualizzazione.

2.0.1 Distribuzione delle Classi

Un aspetto critico nei task di classificazione è il bilanciamento delle etichette. Come evidenziato nella Figura 1, il dataset presenta una distribuzione perfettamente uniforme tra le due classi:

- **Label 0 (Negativo):** 50% del dataset.
- **Label 1 (Positivo):** 50% del dataset.

Questo bilanciamento è ideale in quanto permette di utilizzare l'*accuracy* come metrica di valutazione affidabile, senza la necessità di applicare tecniche di riequilibrio (come *oversampling* o *class weighting*) durante l'addestramento.

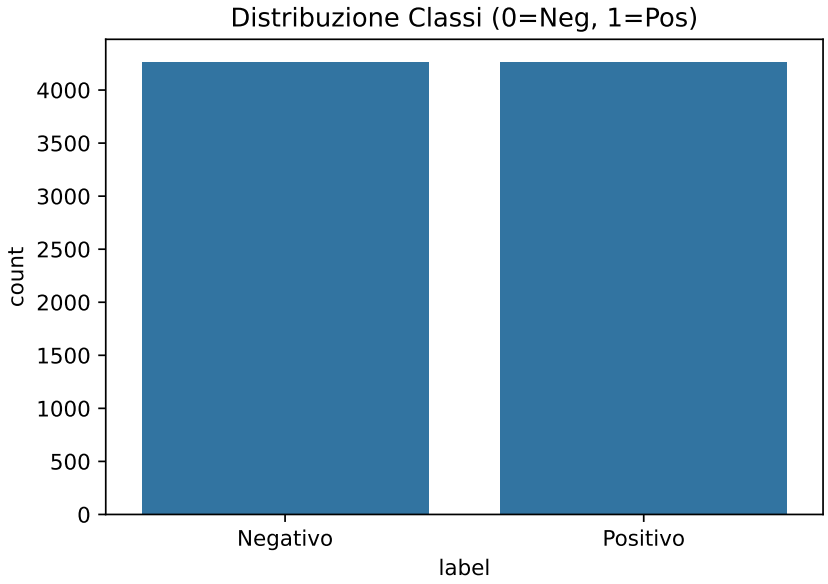


Figura 1: Distribuzione delle classi nel training set. Si nota un perfetto bilanciamento tra recensioni positive e negative.

2.0.2 Analisi della Lunghezza delle Sequenze

È stata inoltre analizzata la lunghezza delle recensioni in termini di numero di parole (token approssimati tramite separazione degli spazi), fattore determinante per la scelta della lunghezza massima dei token (max_length) nel tokenizer di BERT.

Come mostrato nell'istogramma in Figura 2, la distribuzione delle lunghezze appare approssimativamente normale (con una leggera asimmetria positiva). La maggior parte delle recensioni ha una lunghezza compresa tra le 15 e le 30 parole, con pochissimi esempi che superano le 50-60 parole. Ciò suggerisce che le sequenze sono brevi e concise; pertanto,

l'utilizzo di modelli come BERT (che supportano fino a 512 token) non presenterà problemi di troncamento delle informazioni rilevanti.

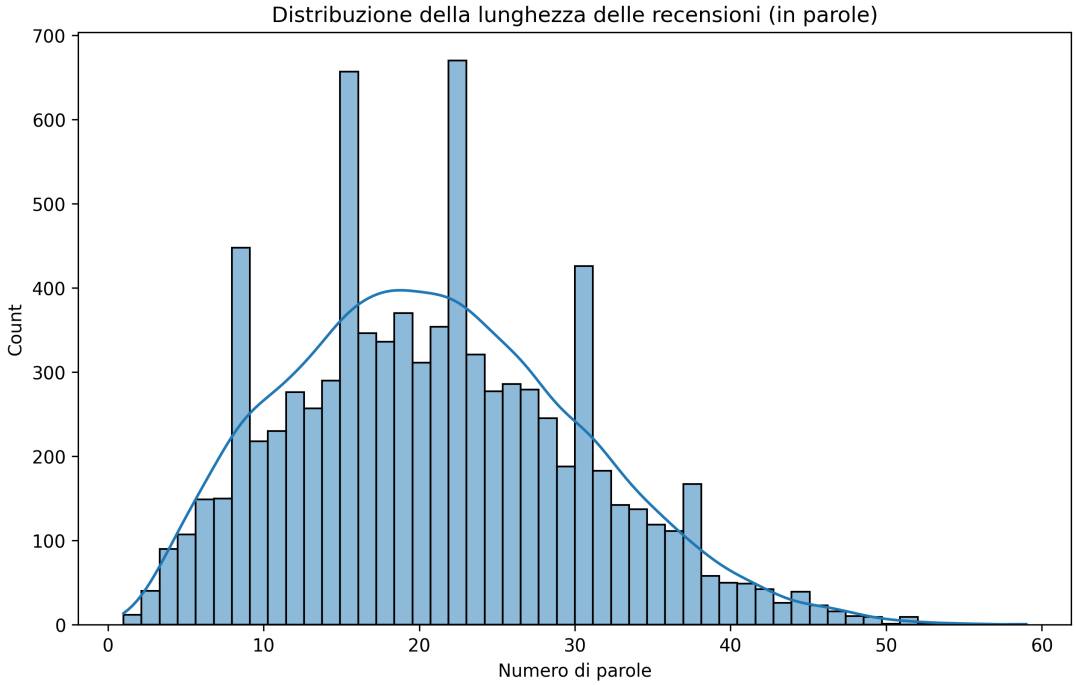


Figura 2: Istogramma della lunghezza delle recensioni. La curva KDE (Kernel Density Estimate) sovrapposta mostra la densità di probabilità della distribuzione.

2.1 Inizializzazione del Modello e Tokenizzazione

Per l'implementazione della pipeline, è stata selezionata l'architettura **DistilBERT** (`distilbert-base-`
Tale scelta progettuale è motivata dall'efficienza del modello che, grazie al processo di *knowledge distillation*, riduce il numero di parametri del 40% rispetto a BERT-base, mantenendo circa il 97% delle performance.

2.1.1 Analisi del Processo di Tokenizzazione

Utilizzando la classe `AutoTokenizer`, le frasi del dataset sono state convertite in tensori pronti per l'input. Dall'analisi dell'output intermedio emergono tre componenti fondamentali gestiti automaticamente:

- **Token ID:** La mappatura delle parole (o subwords) in indici numerici.
- **Token Speciali:** L'aggiunta automatica dei token `[CLS]` (ID 101) all'inizio della sequenza, fondamentale per i task di classificazione, e `[SEP]` (ID 102) al termine.
- **Attention Mask:** Un tensore binario che permette al modello di distinguere i token informativi dai token di *padding* aggiunti per uniformare la lunghezza del batch.

2.1.2 Analisi dell'Output (Hidden States)

Il passaggio dei dati attraverso il modello (`AutoModel`) ha prodotto il tensore `last_hidden_state`. L'ispezione della sua forma, risultata pari a $(2, 52, 768)$, conferma la corretta propagazione del segnale:

$$Shape = (Batch_Size, Sequence_Length, Hidden_Size) \tag{1}$$

Questo indica che per ciascuna delle 2 frasi e per ognuno dei 52 token della sequenza, DistilBERT ha generato un vettore di *contextualized embedding* a 768 dimensioni. Tali vettori rappresentano la comprensione semantica profonda del testo e costituiscono l'input per le teste di classificazione successive.

2.2 Costruzione della Baseline: Feature Extraction con SVM

Al fine di stabilire una linea di base (*stable baseline*) per valutare i successivi miglioramenti ottenuti tramite fine-tuning, è stato implementato un approccio ibrido. In questa fase, **DistilBERT** non viene addestrato, ma utilizzato esclusivamente come *feature extractor* statico per alimentare un classificatore lineare tradizionale.

2.2.1 Estrazione delle Rappresentazioni Semantiche

Il dataset tokenizzato è stato processato attraverso il modello pre-addestrato per ottenere gli *hidden states*. Per ottimizzare le risorse computazionali, il calcolo dei gradienti è stato disabilitato (`torch.no_grad()`), trattando i pesi del transformer come "congelati" (*frozen*).

Dall'output dell'ultimo layer (*last_hidden_state*), è stato estratto solamente il vettore corrispondente al primo token della sequenza, ovvero il token speciale `[CLS]` (indice 0). Nell'architettura BERT, l'embedding di questo token è progettato per catturare una rappresentazione condensata e semantica dell'intera frase, rendendolo ideale per compiti di classificazione a livello di sequenza. Il risultato è un dataset trasformato in cui ogni recensione è rappresentata da un vettore denso di dimensione 768.

2.2.2 Addestramento e Valutazione del Classificatore

Sulle feature estratte è stata addestrata una **Support Vector Machine (SVM)** con kernel lineare. La scelta di un classificatore lineare è motivata dall'alta dimensionalità dello spazio delle feature (768 dimensioni), dove i margini di separazione lineari tendono ad essere efficaci.

Le prestazioni della baseline sono state valutate sul set di validazione e di test. La matrice di confusione normalizzata e il report di classificazione evidenziano come, anche senza fine-tuning specifico, le rappresentazioni pre-addestrate di DistilBERT permettano all'SVM di distinguere efficacemente tra recensioni positive e negative, fornendo un solido punto di partenza per i confronti successivi.

L'addestramento del classificatore SVM sulle feature estratte staticamente ha prodotto risultati notevoli, confermando la ricchezza semantica delle rappresentazioni di DistilBERT anche in assenza di fine-tuning specifico per il task.

Le metriche ottenute sono le seguenti:

- **Validation Accuracy:** 81.89%
- **Test Accuracy (Baseline):** 80.68%

È importante osservare due aspetti fondamentali:

1. **Generalizzazione:** Il divario estremamente ridotto tra le prestazioni in validazione e quelle sul test set (circa 1.2%) indica che il classificatore lineare ha una buona capacità di generalizzazione e non soffre di *overfitting* significativo.

2. **Soglia di Riferimento:** L'accuratezza dell'80.68% sul test set definisce ufficialmente la nostra **Strong Baseline**. Questo valore rappresenta il "minimo sindacale": le tecniche di fine-tuning successive (come Partial FT o LoRA), che richiedono risorse computazionali maggiori, dovranno dimostrare di poter superare significativamente questa soglia per giustificare il loro utilizzo.

2.3 Full Fine-Tuning di DistilBERT

Dopo aver stabilito una baseline solida con SVM, si è proceduto con il *Full Fine-Tuning* del modello DistilBERT. A differenza dell'approccio precedente di *Feature Extraction*, in questa fase tutti i pesi del modello (sia il corpo del transformer che la testa di classificazione) sono stati resi addestrabili e aggiornati tramite retropropagazione.

2.3.1 Setup Sperimentale

L'architettura è stata istanziata utilizzando la classe `AutoModelForSequenceClassification`, che appende automaticamente un layer denso (con dimensione di output pari a 2) sopra il token `[CLS]`. La gestione del training è stata affidata alla classe `Trainer` di HuggingFace, configurata con le seguenti componenti chiave:

- **Data Collator:** È stato impiegato un `DataCollatorWithPadding` per applicare un *padding dinamico*. A differenza del padding statico (dove tutti i batch hanno lunghezza fissa pari a max_seq_len), questa tecnica adatta la lunghezza del tensore alla sequenza più lunga presente nello specifico batch, ottimizzando notevolmente l'uso della VRAM e i tempi di calcolo.

- **Metriche di Valutazione:** Oltre all'accuratezza, la funzione di valutazione calcola precision, recall e F1-score. Data la natura binaria del problema, è stata utilizzata la media `'binary'` per monitorare specificamente la capacità del modello di identificare la classe positiva.

2.3.2 Iperparametri e Training Strategy

Il training è stato eseguito per 2 epoche con un learning rate di $2e^{-5}$, valore conservativo tipico per il fine-tuning di modelli BERT-based al fine di evitare il "catastrophic forgetting" delle conoscenze pre-addestrate. Per massimizzare le prestazioni hardware su GPU T4, è stata abilitata la *Mixed Precision* (FP16). La strategia di salvataggio (*checkpointing*) è stata configurata per monitorare la metrica `eval_f1`, garantendo che il modello finale salvato (`load_best_model_at_end=True`) fosse quello con il miglior bilanciamento precision-recall, indipendentemente dall'eventuale overfitting nelle epoche successive.

2.3.3 Analisi dei Risultati

Il processo di fine-tuning ha prodotto i seguenti risultati sul set di validazione:

- **Validation Loss:** 0.387
- **Accuracy:** 86.02%
- **F1-Score:** 86.06%

L'analisi delle curve di apprendimento (registrate tramite *Weights & Biases*) evidenzia un fenomeno interessante: verso la fine della seconda epoca, la *Validation Loss* inizia a divergere leggermente rispetto alla *Training Loss*, segnalando l'insorgere di un lieve overfitting. Tuttavia, grazie alla strategia di *early stopping* implicita nel salvataggio del miglior modello, le prestazioni finali risultano robuste.

Con un'accuratezza dell'86%, il Full Fine-Tuning supera nettamente la baseline SVM (81.89%), confermando che l'adattamento dei pesi interni del transformer permette di catturare sfumature linguistiche più complesse rispetto alla sola estrazione statica delle feature.



(a) Train Loss (con exponential moving average di weights and biases)



(b) Eval Loss (con exponential moving average di weights and biases)

3 Parameter-Efficient Fine-Tuning (PEFT) con LoRA

Per mitigare l'elevato costo computazionale del *Full Fine-Tuning*, nel terzo esperimento è stata adottata la tecnica **LoRA** (Low-Rank Adaptation).

3.1 Fondamenti Teorici

LoRA si basa sull'ipotesi che l'adattamento di un modello pre-addestrato a un nuovo task avvenga in un sottospazio intrinseco di dimensione ridotta. Invece di aggiornare tutti i parametri della matrice dei pesi pre-addestrata $W_0 \in R^{d\times k}$, LoRA congela W_0 e inietta coppie di matrici di rango ridotto addestrabili A e B, tali che l'aggiornamento dei pesi ΔW sia rappresentato come:

$$W = W_0 + \Delta W = W_0 + BA \tag{2}$$

dove $B \in R^{d\times r}$ e $A \in R^{r\times k}$ con $r \ll \min(d, k)$. Questo approccio permette di ridurre drasticamente il numero di parametri addestrabili, mantenendo inalterata l'architettura durante l'inferenza (le matrici BA possono essere fuse in W_0).

3.2 Configurazione Sperimentale e Risultati

Sono state testate due diverse configurazioni per valutare l'impatto del rango r e dei moduli target sulle prestazioni.

3.2.1 Configurazione FT2: Basic LoRA

Nel primo tentativo è stata applicata una configurazione conservativa:

- **Iperparametri:** Rank $r = 8$, Scaling factor $\alpha = 32$.
- **Target Modules:** L'adattamento è stato limitato alle sole matrici di proiezione `query` (W_Q) e `value` (W_V) del meccanismo di self-attention.

Nonostante l'efficienza estrema, le prestazioni (Accuracy: $\sim 81.6\%$) sono risultate inferiori rispetto al Full Fine-Tuning. Ciò suggerisce che limitare l'adattamento alle sole componenti Q e V non fornisce al modello sufficiente capacità espressiva per eguagliare lo stato dell'arte sul task specifico.

3.2.2 Configurazione FT3: Optimized LoRA

Per colmare il gap prestazionale, la seconda configurazione è stata potenziata agendo su tre fronti:

1. **Aumento della Capacità:** Il rank r è stato raddoppiato a 16 e, fattore cruciale, i moduli target sono stati estesi includendo la matrice di output dell'attenzione (`out_lin`). Questo permette al modello di ricalibrare non solo come "osservare" i token, ma anche come proiettare il risultato dell'attenzione verso i layer successivi.
2. **Training Schedule:** Il numero di epoche è stato portato a 4 e il learning rate aumentato a $3e^{-4}$ (un ordine di grandezza superiore rispetto al Full FT), prassi comune per LoRA che beneficia di aggiornamenti più aggressivi su parametri inizializzati a zero.
3. **Scheduler:** È stato introdotto un *Cosine Scheduler* con *Warmup* del 10% per stabilizzare le prime fasi dell'addestramento.

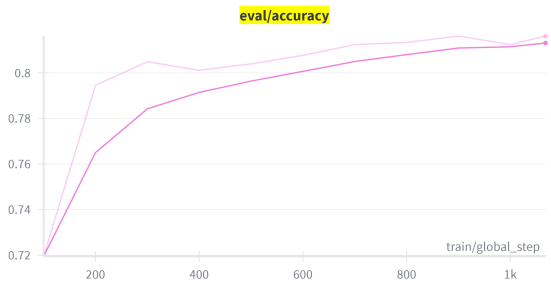
3.2.3 Analisi Comparativa

La configurazione FT3 ha addestrato solamente l'**1.52%** dei parametri totali (circa 1 milione su 66 milioni). Ciononostante, ha raggiunto un F1-score di $\sim 84.7\%$, risultato comparabile al Full Fine-Tuning.

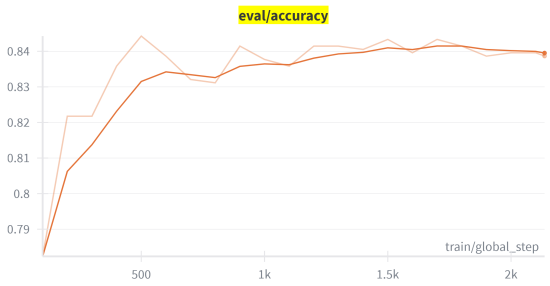
Questo esperimento conferma che, targettizzando i moduli corretti (inclusa la proiezione di output) e utilizzando un rank adeguato, LoRA è in grado di ottenere prestazioni "near state-of-the-art" con un risparmio di memoria e storage di oltre il 98%.

3.3 Ottimizzazione Bayesiana degli Iperparametri

Dopo una prima fase esplorativa condotta tramite *Random Search*, che ha permesso di identificare le regioni promettenti dello spazio degli iperparametri (con F1-score stabile tra 0.82 e 0.85), è stata implementata una strategia di *Ottimizzazione Bayesiana*. L'obiettivo di questa fase è stato raffinare la configurazione di **DistilBERT + LoRA** per



(a) Eval Accuracy FT2 (con exponential moving average di WandB)



(b) Eval accuracy FT3 (con exponential moving average di WandB)

massimizzare la metrica `eval_f1`, cercando di colmare il gap prestazionale con il Full Fine-Tuning riducendo al contempo la varianza dei risultati.

3.3.1 Spazio di Ricerca e Configurazione

Lo sweep bayesiano è stato configurato per esplorare un range di parametri più ristretto e mirato rispetto alla fase precedente:

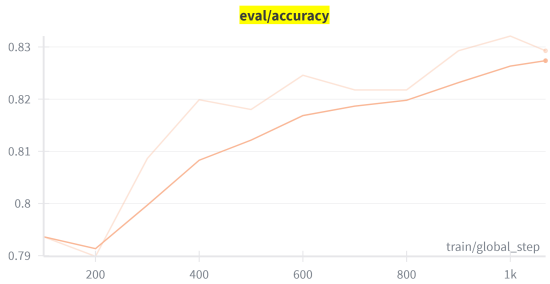
- **Learning Rate:** Intervallo continuo $[1e^{-4}, 4e^{-4}]$. Valori più alti rispetto al fine-tuning standard sono risultati benefici per l'addestramento degli adapter.
- **LoRA Rank (r):** Valori discreti $\{4, 8\}$. L'analisi preliminare ha suggerito che rank elevati ($r = 16$) non portavano benefici significativi tali da giustificare l'aumento dei parametri.
- **Dropout:** Intervallo $[0.08, 0.16]$ per migliorare la regolarizzazione.
- **Weight Decay:** Valori $\{0.0, 0.01\}$ per testare l'impatto della regolarizzazione L2 sui pesi degli adapter.
- **Scheduler:** Utilizzo di un *Cosine Annealing* con warmup del 10%.

3.3.2 Analisi dei Risultati

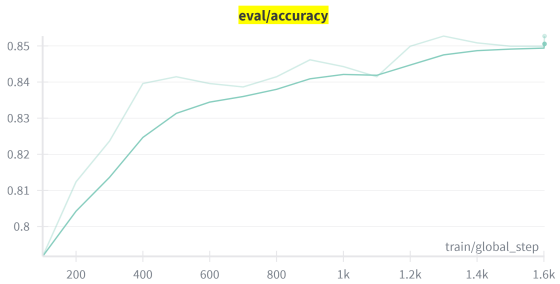
L'ottimizzazione ha prodotto 12 run sperimentali. I risultati migliori confermano la competitività dell'approccio PEFT:

- **Best Run:** Accuratezza $\approx 85.3\%$, F1-score $\approx 85.6\%$.
- **Efficienza:** Questi risultati sono stati ottenuti addestrando meno dell'1.5% dei parametri totali del modello.

Il confronto diretto evidenzia come una configurazione LoRA ottimizzata (Rank 8, LR $\approx 3e^{-4}$) raggiunga prestazioni molto vicine al Full Fine-Tuning (86.0%), offrendo però vantaggi significativi in termini di modularità e impatto sulla memoria.



(a) Un modello con accuracy lievemente più bassa, nel primo sweep



(b) Modello con accuracy alta nel secondo sweep

4 Adattamento tramite Prompt Tuning

Come ulteriore strategia di *Parameter-Efficient Fine-Tuning* (PEFT), è stato testato il **Prompt Tuning**. A differenza di LoRA, questa tecnica non modifica i pesi interni del transformer, ma introduce una sequenza di vettori addestrabili, denominati *virtual tokens* o *soft prompts*, che vengono concatenati all'embedding dell'input originale. Durante la fase di addestramento, solo questi parametri aggiuntivi e la testa di classificazione vengono aggiornati, mantenendo l'intero backbone del modello completamente congelato.

4.1 Configurazione e Iperparametri

Il modello è stato configurato tramite la libreria `peft` utilizzando una `PromptTuningConfig` con i seguenti parametri tecnici:

- **Virtual Tokens:** Sono stati introdotti 20 token virtuali. Questi agiscono come un'istruzione appresa dal modello per guidare l'estrazione delle feature verso il task di *sentiment analysis*.
- **Learning Rate:** È stata adottata una learning rate significativamente più elevata (2×10^{-2}) rispetto al fine-tuning standard. Questo è necessario poiché il gradiente deve agire su un numero molto ridotto di parametri per influenzare l'intero processo decisionale del modello.
- **Training Schedule:** Il modello è stato addestrato per 6 epoche, compensando la minore capacità di adattamento istantaneo rispetto a tecniche che intervengono sui layer interni.

4.2 Analisi Critica dei Risultati

I risultati ottenuti sul set di validazione riportano una **Accuracy del 81.89%** e un **F1-score del 82.28%**.

Dal punto di vista tecnico, si osserva che il Prompt Tuning ha prodotto prestazioni inferiori sia rispetto al Full Fine-Tuning che a LoRA. Tale divergenza è attribuibile a due fattori principali:

1. **Model Scaling:** La letteratura scientifica evidenzia come il Prompt Tuning diventi competitivo con il fine-tuning classico solo all'aumentare delle dimensioni del modello (solitamente oltre i miliardi di parametri). Con un modello "piccolo" come DistilBERT, la capacità latente congelata non è sufficiente per essere pilotata efficacemente dai soli token virtuali.

2. **Dataset Size:** La natura limitata del dataset *Rotten Tomatoes* non favorisce l'apprendimento di *soft prompts* robusti, che traggono maggior beneficio da ampi regimi di dati.

In conclusione, sebbene il Prompt Tuning rappresenti la strategia più estrema in termini di risparmio di parametri, per architetture di scala ridotta LoRA si conferma il miglior compromesso tra efficienza e accuratezza.

5 Strategie di Congelamento Parziale e Confronto Finale

L'ultima fase della sperimentazione ha esplorato metodologie di *Partial Fine-Tuning* e l'estensione dell'approccio PEFT a modelli di scala maggiore, culminando in una valutazione comparativa sul set di test indipendente.

5.1 Fine-Tuning Selettivo di DistilBERT

Sono stati implementati diversi schemi di congelamento per analizzare l'impatto dei singoli componenti sulle prestazioni:

- **Partial FT (FT5):** Sono stati congelati 5 dei 6 layer transformer, mantenendo addestrabili solo l'ultimo layer, gli embedding, il pre-classifier e la testa di classificazione. Con circa l'11% di parametri addestrabili, il modello ha raggiunto una *Validation Accuracy* del 85.65%.
- **Classifier Only (FT6):** È stata addestrata esclusivamente la testa di classificazione lineare, congelando l'intero backbone di DistilBERT. Questo approccio ha prodotto risultati inferiori ($Acc \approx 81.33\%$), simili alla baseline SVM.

5.2 BERT Base e LoRA

Per valutare la scalabilità delle tecniche PEFT, è stata applicata la configurazione **LoRA** (*rank* $r = 8$, $\alpha = 32$) al modello **BERT-base-uncased** (110M parametri). L'adattamento è stato focalizzato sulle matrici di *Query* e *Value* dell'attenzione. Nonostante la maggiore complessità del modello base, la percentuale di parametri addestrabili è rimasta estremamente contenuta, ottenendo un F1-score di validazione del 86.31%.

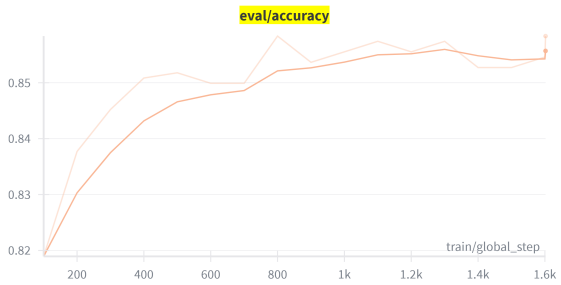
5.3 Valutazione Comparativa sul Test Set

Infine, i migliori modelli salvati sono stati valutati sul **Test Set** per misurare la capacità di generalizzazione reale. I risultati finali sono riportati nella Tabella 1.

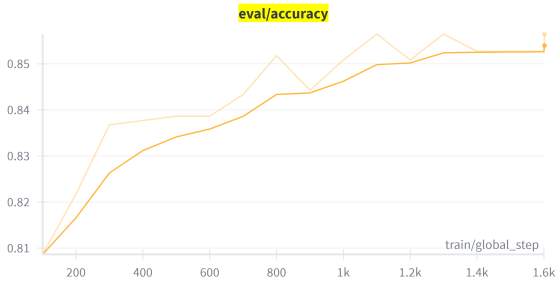
Modello	**Accuracy**	**F1-Score (Weighted)**
DistilBERT (Partial FT - FT5)	0.8471	0.8471
BERT Base (LoRA - r=8)	0.8443	0.8442

Tabella 1: Confronto delle prestazioni finali sul Test Set.

Dall'analisi emerge un dato tecnico rilevante: il modello **DistilBERT con Fine-Tuning parziale** ha mostrato prestazioni leggermente superiori a **BERT Base + LoRA** sul test set. Questo suggerisce che, per task di complessità moderata su dataset di dimensioni ridotte come *Rotten Tomatoes*, un adattamento mirato dell'ultimo layer di un modello distillato può essere più efficace e computazionalmente efficiente rispetto all'uso di adapter su architetture più pesanti.



(a) Eval Accuracy di Bert



(b) Eval accuracy di uno dei modelli DistilBert esaminati (FT5)